ECG ARRHYTHMIA BEAT CLASSIFICATION USING A CONVOLUTIONAL BIDIRECTIONAL LSTM DEPLOYED AS A TOOL FOR A RETRIEVAL-AUGMENTED LLM AGENT

A project report submitted in partial fulfilment of the requirements

for the award of the degree of

MASTER OF SCIENCE IN DATA SCIENCE

Submitted by

PRIYATOSH SATPATI

Registration Number: 25113170096

priyatoshsatpatiofficial@gmail.com

Under the guidance of

MR. G. SWAMINATHAN

SASTRA DEEMED TO BE UNIVERSITY

Thanjavur, Tamil Nadu, India

Fourth Semester · November 2026

BONAFIDE CERTIFICATE

This is to certify that the project report titled **"ECG Arrhythmia Beat Classification Using a Convolutional Bidirectional LSTM Deployed as a Tool for a Retrieval-Augmented LLM Agent"** is a bonafide record of the work carried out by **PRIYATOSH SATPATI** (Registration Number **25113170096**) in partial fulfilment of the requirements for the award of the degree of Master of Science in Data Science of SASTRA Deemed to be University, during the fourth semester of the programme, under my guidance and supervision.

Signature of the Guide Date:
MR. G. SWAMINATHAN
Project Guide and Reporting Manager
SAP ID: C2385

DECLARATION

I, **PRIYATOSH SATPATI** (Registration Number **25113170096**), declare that the project report titled **"ECG Arrhythmia Beat Classification Using a Convolutional Bidirectional LSTM Deployed as a Tool for a Retrieval-Augmented LLM Agent"** submitted to SASTRA Deemed to be University is a record of original work done by me under the guidance of Mr. G. Swaminathan. The source code, experiments, results and text are my own work. All datasets, publications and software used are acknowledged and cited in the References. The project was carried out at SASTRA Deemed to be University. This report does not contain any confidential information or the name of any customer, and it has not been submitted for the award of any other degree or diploma.

Place:
Date:

Signature of the Student
PRIYATOSH SATPATI
priyatoshsatpatiofficial@gmail.com

ACKNOWLEDGEMENT

I thank my guide and reporting manager, **Mr. G. Swaminathan**, for reviewing the scope, methodology and results of this project and for his guidance throughout. I thank the faculty of the Master of Science in Data Science programme at SASTRA Deemed to be University for the coursework this project builds on.

I acknowledge the creators and maintainers of the MIT-BIH Arrhythmia Database and PhysioNet, whose openly available, expert-annotated recordings made this work possible, and the developers of the open-source libraries listed in the References.

PRIYATOSH SATPATI

ABSTRACT

Cardiac arrhythmias are usually identified from long ambulatory electrocardiogram (ECG) recordings, where manual beat-by-beat review is slow and error-prone. Many published deep-learning heartbeat classifiers report near-perfect accuracy, but they often split beats randomly, so beats from the same patient appear in both training and test data and the reported performance does not carry over to new patients. This project builds a heartbeat classifier that is evaluated on unseen patients and makes it usable through a conversational assistant.

Single-lead recordings from the MIT-BIH Arrhythmia Database are band-pass filtered, segmented around each R-peak and grouped into the AAMI classes normal (N), supraventricular ectopic (S) and ventricular ectopic (V). A compact convolutional bidirectional long short-term memory (Conv-BiLSTM) network of about 50,000 parameters combines beat morphology with rhythm features expressed relative to each patient's own heart rate. Model selection uses patients held out from training, and final testing uses 22 patients never seen during development, reported with patient-level bootstrap confidence intervals and compared against baselines.

On the unseen patients the model reaches 94.5% accuracy and a macro-F1 score of 0.65, against 0.53 for a rhythm-only baseline. It detects 80% of ventricular beats with 85% precision, and its probabilities are well calibrated. Supraventricular beats remain difficult, and this limitation is reported explicitly.

The model is deployed in a FastAPI web service with an ECG signal-quality check. An OpenAI language model acts as the agentic core: it decides when to call the classifier as a tool and when to retrieve reference documents from a persistent Chroma vector database (retrieval-augmented generation), with a Gemini model as automatic fallback. Answers about an uploaded ECG therefore rest on both the measurement and cited sources. The system is verified by 119 automated tests, including live tests against the real language-model APIs.

Keywords: electrocardiogram, arrhythmia, heartbeat classification, Conv-BiLSTM, inter-patient evaluation, MIT-BIH, retrieval-augmented generation, LLM tool calling, Chroma, FastAPI.

TABLE OF CONTENTS

1. Introduction
2. Literature Review
3. Dataset and Data Understanding
4. Methodology
5. System Design
6. Implementation
7. Testing and Validation
8. Results and Discussion
9. Limitations and Ethical Considerations
10. Conclusion and Future Work
References
Appendix A: How to Reproduce
Appendix B: Complete Test Case Catalogue

LIST OF FIGURES

Figure 1: Conv-BiLSTM architecture with waveform and rhythm branches
Figure 2: Training and validation curves; the dashed line marks the kept checkpoint
Figure 3: System architecture
Figure 4: Entity relationship diagram (crow's foot notation)
Figure 5: Request flow for a question about an uploaded ECG
Figure 6: Held-out macro-F1 of the model and the baselines
Figure 7: Confusion matrix on the DS2 test patients
Figure 8: Reliability diagram of the predicted-class confidence on DS2
Figure 9: Model output on record 233 compared with cardiologist labels

LIST OF TABLES

Table 1: Mapping of MIT-BIH beat symbols to AAMI EC57 classes [4]
Table 2: Patient-wise split and beat counts per class
Table 3: Training configuration
Table 4: Model versions and why each was kept or rejected
Table 5: REST endpoints
Table 6: Software used
Table 7: Main modules and files
Table 8: Test suites and results (119 of 119 passed)
Table 9: Selected model accuracy and robustness test cases
Table 10: Model performance test cases (measured value in brackets)
Table 11: Selected API and integration test cases
Table 12: Live end-to-end test cases with real LLMs
Table 13: Injected faults and the tests that detected them
Table 14: Performance on the 22 DS2 test patients at annotated R-peaks (95% patient-bootstrap CI)
Table 15: Per-class results on DS2
Table 16: Speed and resources on an Intel i5-7200U laptop CPU without GPU
Table 17: All automated test cases

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AAMI	Association for the Advancement of Medical Instrumentation
API	Application Programming Interface
AUC	Area Under the ROC Curve
BiLSTM	Bidirectional Long Short-Term Memory
CI	Confidence Interval
CNN / Conv1D	(One-dimensional) Convolutional Neural Network
DS1 / DS2	Training / test record sets of de Chazal et al. [3]
ECE	Expected Calibration Error
ECG	Electrocardiogram
ERD	Entity Relationship Diagram
F / Q	Fusion / unclassifiable (paced) beats
LLM	Large Language Model
MLII	Modified limb lead II
N / S / V	Normal / supraventricular ectopic / ventricular ectopic beats
PPV	Positive Predictive Value (precision)
PVC	Premature Ventricular Contraction
RAG	Retrieval-Augmented Generation
ROC	Receiver Operating Characteristic
RR interval	Time between consecutive R-peaks
Se	Sensitivity (recall)
SDNN	Standard deviation of normal-to-normal RR intervals

CHAPTER 1: INTRODUCTION

1.1 Background

The electrocardiogram (ECG) records the electrical activity of the heart and is the primary tool for detecting arrhythmias, i.e. disturbances of the heart's rhythm. Many arrhythmias are intermittent, so patients are monitored with ambulatory (Holter) recorders for 24 hours or longer. A single day contains roughly 100,000 heartbeats, and reviewing them manually is slow, tiring and prone to error. Automatic heartbeat classification therefore has clear clinical value as a screening aid.

Two developments shape this project. First, deep learning models that combine convolutional and recurrent layers can learn beat morphology directly from the raw signal [8]–[11]. Second, large language models (LLMs) can call external tools and ground their answers in retrieved documents [14]–[16]. Combining the two allows a clinician or researcher to ask questions in natural language about an uploaded ECG and receive an answer based on an actual measurement and on cited reference material, rather than on the LLM's general knowledge.

1.2 Problem Statement

Published ECG classifiers frequently report accuracies of 98–99%, but many of them split heartbeats randomly into training and test sets. Because beats from one patient are highly similar, the model is then tested on people it has already seen, and the result does not describe its performance on a new patient [3], [13]. In addition, a trained classifier on its own is difficult for non-programmers to use and gives no context for its output.

The problem addressed is therefore: **to build an ECG heartbeat classifier whose performance is measured on patients it has never seen, to report that performance honestly with its uncertainty and limitations, and to make the classifier usable through a conversational agent whose answers are grounded in the model's output and in a searchable knowledge base.**

1.3 Objectives

1. Explore the MIT-BIH Arrhythmia Database and build a preprocessing pipeline (filtering, R-peak detection, beat segmentation, rhythm features) shared by training and deployment.
2. Train a compact Conv-BiLSTM classifier for the AAMI classes N, S and V that runs on an ordinary laptop CPU.
3. Evaluate it with a strict inter-patient protocol, baselines, patient-level confidence intervals, end-to-end detection, calibration and speed measurements.

4. Deploy the model in a FastAPI service with a persistent Chroma vector store for retrieval-augmented generation (RAG).
5. Expose the classifier and the knowledge base as tools to an LLM agent whose agentic core is an OpenAI model, with a Gemini fallback, and as a separate tools API usable without an LLM.
6. Verify the system with automated unit, API, integration, model-accuracy, model-performance and live LLM tests.

1.4 Scope

The system classifies individual beats of a single-lead ECG into N, S and V. Fusion (F) and paced or unclassifiable (Q) beats are outside the trained classes; how the model labels them is reported. The system is a research and education tool and is **not** a medical device. The LLM (OpenAI gpt-4o-mini, with Gemini 2.5 Flash as fallback) is an external service and is not part of the trained model.

1.5 Organisation of the Report

Chapter 2 reviews related work. Chapter 3 describes the dataset. Chapter 4 presents the methodology, Chapter 5 the system design including the entity relationship diagram, and Chapter 6 the implementation. Chapter 7 describes the testing strategy and test cases, and Chapter 8 the results. Chapter 9 discusses limitations and ethics, and Chapter 10 concludes.

CHAPTER 2: LITERATURE REVIEW

2.1 Data and Evaluation Standards

The MIT-BIH Arrhythmia Database [1], distributed through PhysioNet [2], is the most widely used benchmark for beat classification. The AAMI EC57 standard [4] groups its beat annotations into five classes (N, S, V, F, Q) and defines how sensitivity and positive predictivity are reported. De Chazal et al. [3] proposed splitting the 44 non-paced records into two sets of 22 patients (DS1 for training, DS2 for testing) so that no patient contributes to both. This *inter-patient* protocol has become the reference for realistic evaluation.

2.2 Feature-Based Methods

Early systems detected the QRS complex with the Pan–Tompkins algorithm [5] and classified beats from hand-crafted morphology and RR-interval features with linear discriminants [3] or ensembles of classifiers [12]. Llamedo and Martínez [13] showed that selecting features for generalisation across databases improves performance on unseen patients, and that RR-interval features normalised to the patient's own rhythm are among the most robust inputs.

2.3 Deep Learning Methods

Kiranyaz et al. [8] used one-dimensional CNNs on raw beats, Acharya et al. [9] trained a nine-layer CNN, and Yildirim [11] used bidirectional LSTMs [6], [7] on wavelet sequences. Hannun et al. [10] trained a deep network on a very large single-lead dataset and reported cardiologist-level performance on rhythm classes. Many deep-learning papers on MIT-BIH, however, use random beat-level splits; their very high accuracies are not comparable to inter-patient results, which are substantially lower, especially for supraventricular beats [3], [13].

2.4 Calibration and Uncertainty

Guo et al. [21] showed that modern neural networks are often over-confident and proposed the expected calibration error (ECE) to measure the gap between confidence and accuracy. The Brier score [24] measures the overall quality of probabilistic predictions. Because beats from one patient are correlated, confidence intervals should be computed by resampling patients rather than beats, using the bootstrap [20].

2.5 LLM Agents, Tool Use and RAG

Retrieval-augmented generation [14] grounds an LLM's answer in passages retrieved from an external store, reducing unsupported statements and allowing sources to be cited. ReAct [15] interleaves reasoning with tool calls, and Toolformer [16] showed that

language models can learn when to call external APIs. Dense retrieval relies on sentence embeddings such as Sentence-BERT [17]; MiniLM [18] provides a small embedding model that runs on a CPU.

2.6 Research Gap

Few works combine (i) a beat classifier evaluated with a strict inter-patient protocol and patient-level uncertainty, (ii) deployment as a service with tests that guard its accuracy, and (iii) integration as a tool for an LLM agent with retrieval over reference documents. This project addresses that combination with modest hardware and fully open data.

CHAPTER 3: DATASET AND DATA UNDERSTANDING

3.1 MIT-BIH Arrhythmia Database

The database contains 48 half-hour two-channel ambulatory ECG recordings from 47 subjects, digitised at 360 Hz with 11-bit resolution over a 10 mV range [1]. Every beat was annotated independently by two or more cardiologists. The database is available on PhysioNet [35]. This project uses the modified limb lead II (MLII), which is available in all non-paced records. The data is downloaded from PhysioNet [2] with the WFDB Python package and is distributed under the Open Data Commons Attribution License.

3.2 AAMI Class Mapping

Table 1: Mapping of MIT-BIH beat symbols to AAMI EC57 classes [4]

AAMI class	MIT-BIH symbols	Meaning
N	N, L, R, e, j	Normal, left/right bundle branch block, atrial/nodal escape
S	A, a, J, S	Atrial, aberrated atrial, nodal and supraventricular premature beats
V	V, E	Premature ventricular contraction, ventricular escape
F	F	Fusion of ventricular and normal beat (not modelled)
Q	/, f, Q	Paced, fusion of paced and normal, unclassifiable (not modelled)

3.3 Patient-Wise Data Split

The four paced records (102, 104, 107, 217) are excluded. The remaining records follow the inter-patient split of de Chazal et al. [3]. Five DS1 patients are held out as a validation set for early stopping and model selection, so the DS2 test patients are used only once, for the final evaluation (Table 2).

Table 2: Patient-wise split and beat counts per class

Set	Records	Patients	N	S	V	Purpose
Train	101, 108, 109, 112, 114, 115, 116, 119, 122, 203, 207, 208, 209, 215, 220, 223, 230	17	36,436	686	2,936	Fit weights
Validation	106, 118, 124, 201, 205	5	9,412	258	852	Early stopping, model selection
Test (DS2)	100, 103, 105, 111, 113, 117, 121, 123, 200, 202, 210, 212, 213, 214, 219, 221, 222, 228, 231, 232, 233, 234	22	44,240	1,837	3,220	Final evaluation only

3.4 Observations from Exploratory Analysis

- **Severe class imbalance.** Normal beats make up about 90% of all beats, so a model that always predicts N already reaches about 90% accuracy. Accuracy alone is therefore misleading, and macro-F1 is used for model selection.

- **Patient concentration.** Rare classes are concentrated in few patients. In the test set, 1,382 of the 1,837 S beats come from a single record (232), and 373 of the 415 fusion beats available for training come from record 208.
- **Rhythm is informative.** Premature beats arrive early relative to the patient's own rhythm and are often followed by a compensatory pause, which motivates the patient-relative RR features of Section 4.1.
- **Noise and baseline wander.** The power spectrum shows low-frequency baseline wander and high-frequency noise, removed by a 0.5–40 Hz band-pass filter (notebook 01).

CHAPTER 4: METHODOLOGY

4.1 Preprocessing Pipeline

The same module (app/ecg/preprocessing.py) is used by the training notebooks and by the deployed service, so the beats seen at inference are built exactly like the training beats.

1. **Resampling** to 360 Hz, so recordings from devices sampling at, for example, 250 Hz are accepted.
2. **Band-pass filtering** at 0.5–40 Hz with a third-order zero-phase Butterworth filter.
3. **R-peak detection**: annotated peaks during training; at inference a Pan–Tompkins-style detector [5] (derivative, squaring, 150 ms moving-window integration, adaptive threshold, 250 ms refractory period, refinement to the local maximum) or peaks supplied by the caller.
4. **Segmentation**: a 256-sample window per beat, 90 samples (0.25 s) before and 166 samples (0.46 s) after the R-peak, z-score normalised per beat.
5. **Rhythm features**, all relative to the patient's own rhythm: previous and next RR interval divided by the local mean RR of about 10 beats, previous RR divided by the recording's median RR, and the ratio of previous to next RR. Absolute RR values in seconds are deliberately excluded, because they identify individual patients and caused over-fitting in an earlier version.
6. **Signal-quality index**: the excess kurtosis of each 10-second window of the filtered signal. QRS complexes make a clean ECG strongly peaked (median kurtosis about 10 on MIT-BIH), while noise and flat or periodic artefacts score near or below 0 [28]. A recording with more than half of its windows below 1.5 is flagged as unreliable. This check was added after an integration test showed that the detector reports beats in pure noise (Section 7.6).

4.2 Model Architecture

The classifier has two inputs (Figure 1). The beat waveform passes through two convolutional blocks that learn local morphology (QRS width, T-wave shape) and shorten the sequence from 256 to 64 steps, followed by two bidirectional LSTM layers [6], [7] that model the temporal order of the waves. The four rhythm features pass through batch normalisation [22] and a dense layer. Both branches are concatenated and classified by a dense layer with dropout [23] and a three-way softmax. Gaussian noise layers act as data augmentation during training only. The network has 49,571 parameters (0.71 MB).

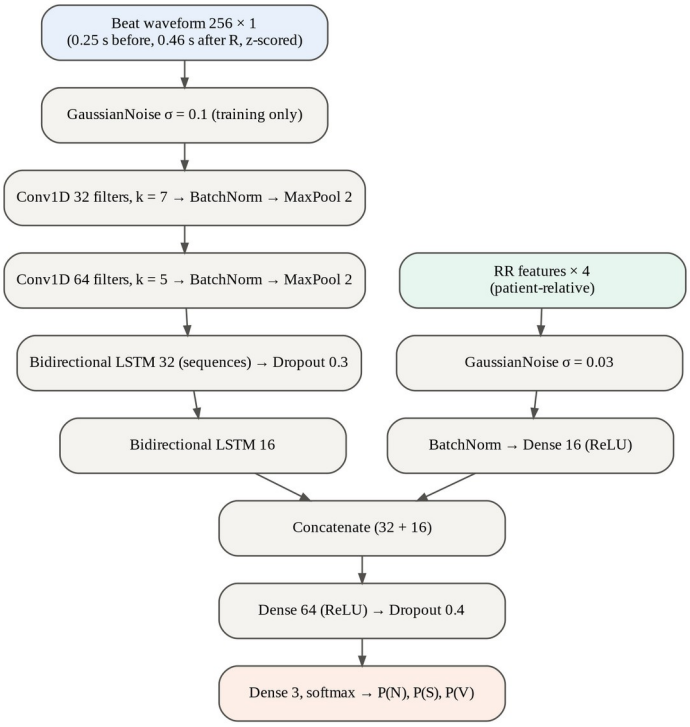


Figure 1: Conv-BiLSTM architecture with waveform and rhythm branches

4.3 Training Procedure

Table 3: Training configuration

Setting	Value	Reason
Optimiser	Adam [19], learning rate 0.001, halved on plateau	Standard, robust default
Batch size / max epochs	512 / 20	Fits a 2-core laptop CPU (about 60 s per epoch)
Loss	Categorical cross-entropy with balanced class weights	Counteracts the 90% majority of N beats
Model selection	Validation macro-F1, early stopping (patience 4)	Accuracy is dominated by N; loss does not reflect rare classes
Augmentation	Gaussian noise σ = 0.1 on beats, 0.03 on RR features	Robustness to noise and unseen morphology
Seed	42	Reproducibility

Training stopped after 8 epochs; the checkpoint from epoch 4 had the best validation macro-F1 (0.679) and was kept (Figure 2). Validation macro-F1 fluctuates between epochs because the validation set contains only five patients; selecting on it, rather than on training loss, avoids keeping a model that has started to over-fit.

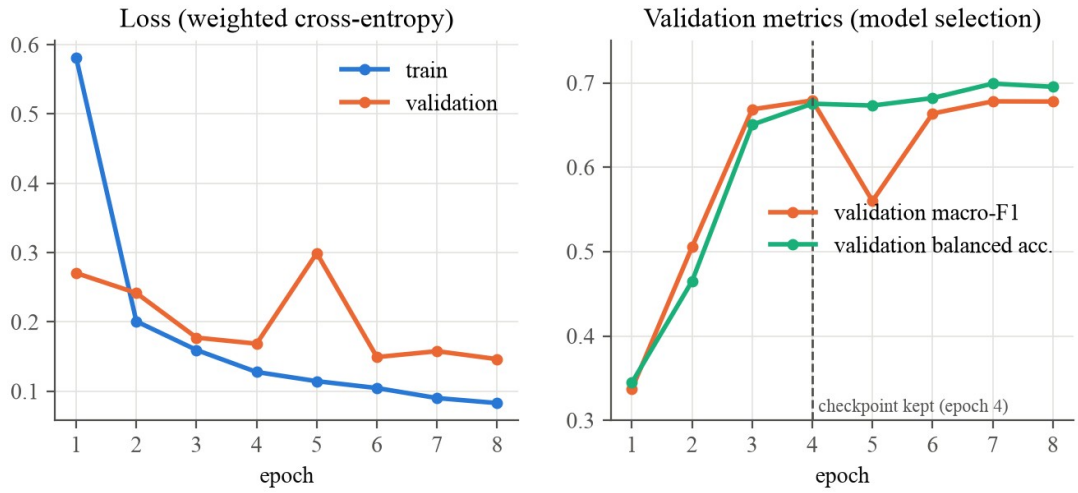


Figure 2: Training and validation curves; the dashed line marks the kept checkpoint

4.4 Evaluation Design

For each class c, with true positives TP, false negatives FN, false positives FP and true negatives TN:

$$\text{Sensitivity } Se = TP / (TP + FN) \qquad PPV = TP / (TP + FP) \qquad \text{Specificity} = TN / (TN + FP)$$

$$F1 = 2 \cdot Se \cdot PPV / (Se + PPV) \qquad \text{macro-}F1 = \text{mean of } F1 \text{ over N, S, V}$$

$$\text{balanced accuracy} = \text{mean of } Se \text{ over N, S, V}$$

- **Baselines**: a majority-class model (always N) and a logistic regression on the four RR features only, fitted on the same training patients.
- **Confidence intervals**: 1,000 bootstrap resamples of the 22 test *patients* [20]; beat-level resampling would understate the uncertainty.
- **End-to-end evaluation**: the deployed R-peak detector replaces the annotations; detected beats are matched to annotations within ±150 ms, as in AAMI EC57 [4].
- **Probability quality**: one-vs-rest ROC AUC, Brier score [24], ECE with 10 bins [21], and selective accuracy at confidence thresholds.
- **Significance**: a paired patient-level bootstrap comparing the macro-F1 of the model and the RR-only baseline on the same resampled patients.

4.5 Model Development History

Table 4: Model versions and why each was kept or rejected

Version	Change	DS2 macro-F1	Outcome
v1	4 classes, random beat-level validation split, absolute and relative RR features	not evaluated	Discarded: validation contained training patients (leakage)
v2	Patient-wise validation, macro-F1 selection, 4 classes (N/S/V/F)	0.458	Rejected: about 11% of normal beats labelled F (memorised patient 208); S Se 0.07

Version	Change	DS2 macro-F1	Outcome
v3	N/S/V only, patient-relative RR features, balanced weights, noise augmentation	0.650	Final model

CHAPTER 5: SYSTEM DESIGN

5.1 Architecture

Figure 3 shows the components. The design separates the **agentic core** from the **tools**. The agent (agent.py) holds the conversation with the LLM: OpenAI gpt-4o-mini decides which tools to call and writes the answer, and Gemini 2.5 Flash takes over automatically if the OpenAI call fails. The tools (tools.py) are plain functions with JSON schemas: ECG analysis by the Conv-BiLSTM, semantic search of the Chroma knowledge base, and listing of uploaded recordings. They know nothing about the LLM.

The FastAPI application exposes this separation as four groups of endpoints: the **Agent API** (/agent) runs the agent loop and returns a step-by-step trace; the **Tools API** (/tools) lists the tool schemas and invokes any tool directly, so another program or agent can use the LSTM exactly as the LLM does; the **ECG** and **Document** endpoints upload recordings and reference documents. All services are created once at start-up (FastAPI lifespan) and shared by the requests.

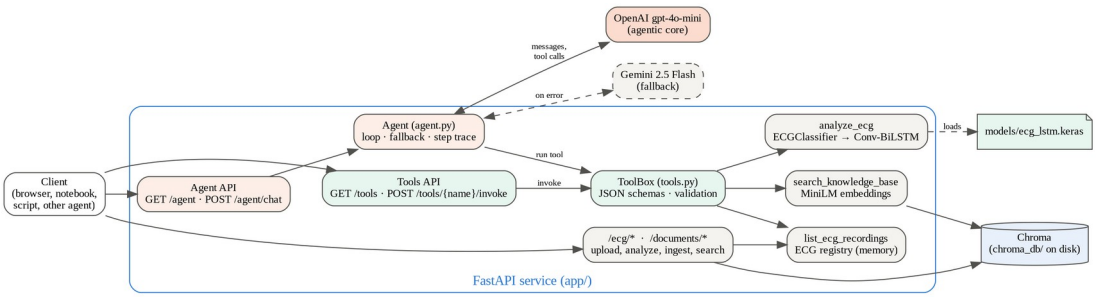


Figure 3: System architecture

5.2 Entity Relationship Diagram

Figure 4 shows the logical data model. Three groups of entities can be distinguished:

- **Training data (grey and green)**: a PATIENT_RECORD has many ANNOTATIONs and is segmented into many BEATs; beats train and evaluate a MODEL_VERSION, whose metadata (classes, hyperparameters, split, metrics) is stored in ecg_lstm_meta.json.
- **Runtime ECG analysis (blue)**: an ECG_UPLOAD is analysed by a MODEL_VERSION into an ANALYSIS_RESULT (heart rate, SDNN, beat counts, signal quality), which lists ABNORMAL_BEATs with time and confidence.
- **Knowledge base and agent (orange and pink)**: a Chroma COLLECTION stores DOCUMENTs, each split into CHUNKs with a 384-dimensional embedding and

the identifier source::index. A CHAT_TURN may refer to an ECG_UPLOAD and makes TOOL_CALLs, which retrieve chunks or run an analysis.

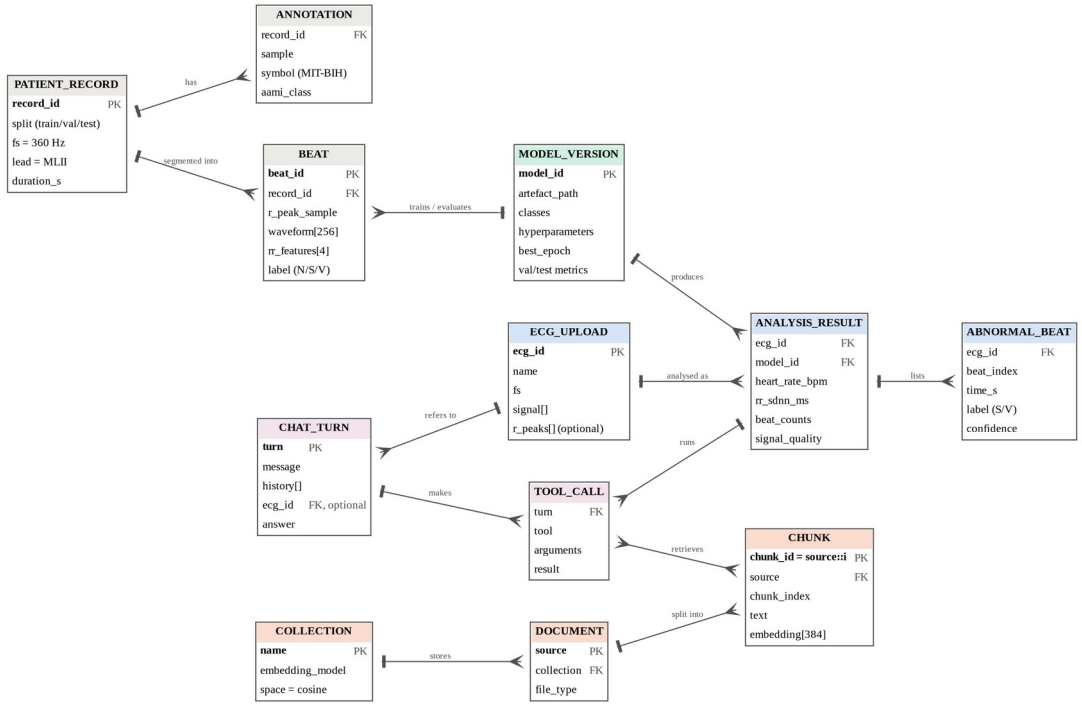


Figure 4: Entity relationship diagram (crow's foot notation)

Physically, chunks, embeddings and collection metadata live in Chroma's on-disk store (chroma_db/), the model and its metadata in models/, and uploaded ECGs in an in-memory registry. The collection records the name of the embedding model that built it, and the service refuses to mix embeddings from different models.

5.3 Request Flow of the Agent

Figure 5 shows a typical question about an uploaded ECG. The agent sends the conversation and the tool schemas to the LLM; the LLM requests tool calls; the agent executes them against the real classifier and vector store and returns the JSON results; the LLM then writes an answer grounded in those results. The loop is limited to six LLM calls. Every LLM decision and tool execution is recorded in a trace with its duration and token usage, which the client receives with the answer. Failures of both providers are returned as HTTP 502 with a configuration hint.

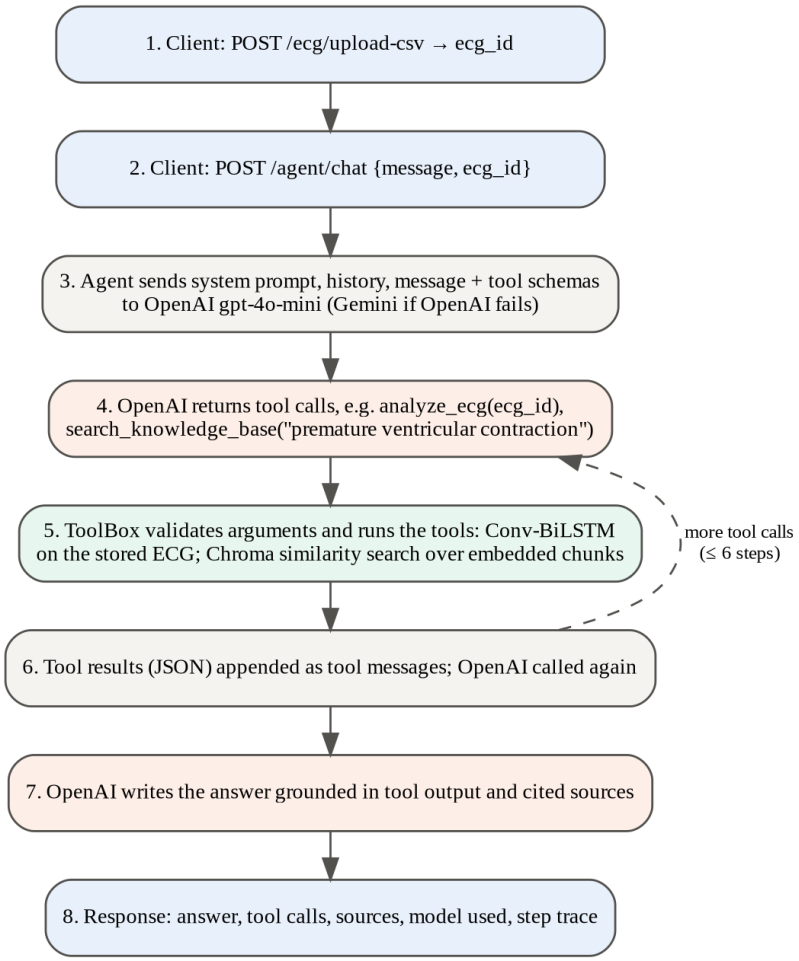


Figure 5: Request flow for a question about an uploaded ECG

5.4 API Endpoints

Table 5: REST endpoints

Method	Path	Purpose
GET	/health	Status, vector collection, model loaded, LLM in use
GET	/agent	Agent configuration: provider, model, fallback, tools, system prompt
POST	/agent/chat	Run the agent: answer, tool calls, sources, model used, step trace
GET	/tools	Tool catalogue with JSON schemas (what the LLM is offered)
GET	/tools/{name}	Schema of one tool
POST	/tools/{name}/invoke	Run a tool directly without an LLM (404, 422, 503 on errors)
POST	/documents/upload	Upload PDF, TXT or Markdown; chunk, embed and store
POST	/documents/text	Ingest raw text under a source name
POST	/documents/search	Semantic search with optional source filter
DELETE	/documents/{source}	Remove all chunks of a document
GET	/documents/count	Number of stored chunks

Method	Path	Purpose
POST	/ecg/upload	Upload an ECG as JSON (signal, sampling rate, optional R-peaks)
POST	/ecg/upload-csv	Upload an ECG as CSV
GET	/ecg/	List uploaded recordings
POST	/ecg/{ecg_id}/analyze	Run the classifier; returns heart rate, counts, abnormal beats, signal quality
POST	/chat	Alias of /agent/chat for existing clients

5.5 RAG Design

Documents are split with a recursive character splitter into chunks of 800 characters with 120 characters of overlap. Chunks are embedded locally with all-MiniLM-L6-v2 [17], [18] (ONNX runtime, no API key) or optionally with an OpenAI-compatible embedding endpoint. Chroma indexes the vectors with HNSW under cosine distance; search results are returned with a similarity score of 1 − distance. Re-uploading a document replaces its chunks instead of duplicating them.

5.6 Agent and LLM Providers

Providers are selected in the configuration file. LLM_PROVIDER=openai uses OPENAI_API_KEY and gpt-4o-mini; gemini uses GEMINI_API_KEY through Google's OpenAI-compatible endpoint; custom accepts any OpenAI-compatible server such as Ollama or vLLM. LLM_FALLBACK_PROVIDER names the provider used when the main one raises an API error (authentication, quota, outage); once switched, the rest of the conversation stays on the fallback. The response states which provider and model produced the answer.

Before a tool runs, its arguments are validated against the tool's JSON schema (required fields, unknown fields, types). For the LLM, every failure becomes a JSON error message the model can react to; for direct API callers, the same failures become HTTP status codes. The system prompt instructs the model to call analyze_ecg instead of guessing, to cite knowledge-base sources, to mention model confidence and limitations, to ignore results flagged by the signal-quality check, and to state that the output is not a diagnosis.

CHAPTER 6: IMPLEMENTATION

6.1 Technology Stack

Table 6: Software used

Component	Library (version)
Language	Python 3.12
Deep learning	TensorFlow 2.21 / Keras 3.15 [26]
Signal processing, data	NumPy 2.5 [30], SciPy 1.18 [29], WFDB 4.3 [27], pandas 3.0
Classical ML and metrics	scikit-learn 1.9 [25]
Web service	FastAPI 0.139 [34], Uvicorn 0.51, Pydantic 2.13
Vector database	ChromaDB 1.5 (persistent client) [32]
Embeddings	all-MiniLM-L6-v2 via ONNX (Chroma default) [18]
LLM (agentic core)	OpenAI gpt-4o-mini via openai 2.45 [31]; fallback Gemini 2.5 Flash [36]
Document parsing	pypdf 6.14, langchain-text-splitters 1.1
Testing	pytest 9.2, FastAPI TestClient, psutil
Notebooks and plots	Jupyter, Matplotlib 3.11
Hardware	Intel Core i5-7200U (2 cores, 4 threads), 16 GB RAM, no GPU

6.2 Project Structure

Table 7: Main modules and files

Path	Responsibility
app/ecg/preprocessing.py	Filtering, R-peak detection, segmentation, RR features, signal quality
app/ecg/mitbih.py	Download, loading and patient splits of MIT-BIH
app/ecg/classifier.py	Inference wrapper used by the API and the agent tool
app/ecg/evaluation.py	AAMI metrics, bootstrap CIs, end-to-end and probability metrics
app/vector_store.py, embeddings.py, ingest.py	Chroma store, embeddings, text extraction and chunking
app/agent.py	Agentic core: LLM loop, provider fallback, step trace, system prompt
app/tools.py	Tool layer: JSON schemas, argument validation, handlers, ECG registry
app/routers/agent.py, tools.py	Agent API and Tools API
app/routers/ecg.py, documents.py, main.py, config.py	ECG and document endpoints, start-up, provider settings
notebooks/01–03	Exploratory analysis, training and validation, LSTM as an LLM tool
scripts/evaluate_model.py, benchmark_model.py	Accuracy report and performance benchmark (reports/)
scripts/test_report.py, make_figures.py	Test report and report figures
scripts/demo_agent.py	Live demonstration of the tools API and the agent against a running server

Path	Responsibility
tests/	119 automated tests (Chapter 7)

6.3 Configuration and Notebooks

All settings (Chroma path and collection, LLM provider, model, fallback and keys, embedding provider, chunking, model paths) are read from a .env file; a template (.env.example) is versioned while the real .env is excluded from version control. Notebook 01 explores the data, notebook 02 trains, validates and tests the model and writes the model, its metadata and the training log, and notebook 03 demonstrates the model as an LLM tool, with and without RAG, both directly and through the REST API.

CHAPTER 7: TESTING AND VALIDATION

7.1 Strategy

Testing follows a layered approach. Fast unit and API tests use fakes for the LLM, the vector store and the classifier, so they run in seconds without a network or a trained model. Model, performance and integration tests run against the trained model and real MIT-BIH data, using only the 22 held-out DS2 patients. Thresholds are set a few points below the measured values, so the tests fail on real regressions rather than on normal variation between machines. In the offline suites the LLM is scripted, so the agent loop can be tested deterministically; the scripted LLM issues real tool calls that run against the real classifier and vector store. A separate live suite, enabled with RUN_LLM_TESTS=1, runs the complete system against the real OpenAI and Gemini APIs. Its assertions do not depend on wording: they check which tools were called with which arguments, that the cited sources exist, and that the numbers in the answer match the classifier's output.

Table 8: Test suites and results (119 of 119 passed)

Suite	Level	Scope	Tests	Passed
test_preprocessing.py	Unit	Signal processing, R-peak detection, RR features, signal quality	25	25
test_ingest.py	Unit	Text extraction (txt, md, pdf) and chunking	5	5
test_vector_store.py	Component	Real Chroma store on disk	5	5
test_agent.py	Unit	LLM tool-calling loop with a scripted LLM	15	15
test_api.py	API	Every endpoint with fakes; validation and error codes	20	20
test_model.py	Model	Correctness, robustness, accuracy floors on DS2	13	13
test_model_performance.py	Performance	Speed, memory, calibration, AUC, error rates, significance	14	14
test_api_integration.py	Integration	Real app: trained model, Chroma, local embeddings	11	11
test_llm_live.py	Live	Real OpenAI agent (Gemini fallback) with LSTM and RAG tools	11	11

7.2 Model Test Cases

Table 9: Selected model accuracy and robustness test cases

ID	Test case	Expected result	Status
TC-M01	Train, validation and test record lists are disjoint; test set equals DS2	No patient in two sets	Pass
TC-M02	API inference path vs training pipeline on record 233	≤ 2% of beat labels differ	Pass
TC-M03	Predictions repeated on the same beats	Identical; probabilities sum to 1	Pass
TC-M04	Signal scaled ×3 and offset +0.5	Beat counts unchanged (±1)	Pass
TC-M05	Record resampled to 250 Hz	Beats, V count and heart rate within 2%, 10%, 2 bpm	Pass

ID	Test case	Expected result	Status
TC-M06	White noise added at 25 dB SNR	V count within 10%, heart rate within 2 bpm	Pass
TC-M07	Record 100 (sinus rhythm)	Heart rate 65–85 bpm, ≥ 90% N	Pass
TC-M08	Record 233 (frequent PVCs)	≥ 70% of annotated V beats reported	Pass
TC-M09	Accuracy floors on DS2	Acc ≥ 0.92, macro-F1 ≥ 0.60, Se N ≥ 0.97, Se V ≥ 0.75, PPV V ≥ 0.78	Pass
TC-M10	Comparison with baselines	Macro-F1 above majority and RR-only baselines	Pass
TC-M11	End-to-end detection on DS2	Detection Se ≥ 0.97, PPV ≥ 0.99	Pass
TC-M12	All 22 DS2 records through signal-quality check	None flagged	Pass
TC-M13	Flat and 3-second inputs	No crash; valid response	Pass

7.3 Model Performance Test Cases

Table 10: Model performance test cases (measured value in brackets)

ID	Test case	Expected result	Status
TC-P01	Model size	< 100k parameters, < 2 MB (49,571; 0.71 MB)	Pass
TC-P02	Batched inference speed (2,048 beats)	< 5 ms per beat (0.447 ms)	Pass
TC-P03	Full 30-minute record through analyze()	< 30 s (1.31 s)	Pass
TC-P04	Batch of 40 vs 40 single predictions	Equal within 1e-4	Pass
TC-P05	Ten repeated analyses of a 5-minute strip	Memory growth < 200 MB (no leak)	Pass
TC-P06	Calibration on DS2	ECE < 0.05, Brier < 0.15 (0.0126; 0.0924)	Pass
TC-P07	ROC AUC per class	V ≥ 0.95, N ≥ 0.88, S ≥ 0.70 (0.9836; 0.9257; 0.7638)	Pass
TC-P08	Selective accuracy at rising confidence	Monotonic; ≥ 0.95 accuracy at ≥ 85% coverage	Pass
TC-P09	Normal beats predicted as V	< 1% (0.33%)	Pass
TC-P10	V specificity	≥ 0.98 (0.990)	Pass
TC-P11	S sensitivity without record 232	≥ 0.30 (0.382)	Pass
TC-P12	V sensitivity per patient	≥ 0.7 in ≥ 75% of patients; none < 0.3	Pass
TC-P13	Bootstrap CI of macro-F1 vs majority baseline	Lower bound > 0.315 (0.544)	Pass
TC-P14	Paired patient bootstrap vs RR-only baseline	Model better in ≥ 90% of resamples	Pass

7.4 API and Integration Test Cases

Table 11: Selected API and integration test cases

ID	Test case	Expected result	Status
TC-A01	GET /health	200, collection name, model loaded flag	Pass
TC-A02	Upload, search and delete a document	Chunks stored, relevant hit first, count back to 0	Pass
TC-A03	Re-ingest a document with the same name	Old chunks replaced, not duplicated	Pass
TC-A04	Unsupported file type or empty text	400	Pass

ID	Test case	Expected result	Status
TC-A05	Signal shorter than 256 samples, fs = 0, top_k = 0	422	Pass
TC-A06	Analyse an unknown ecg_id	404	Pass
TC-A07	Analyse while the model is not loaded	503	Pass
TC-A08	Chat with the LLM provider unreachable	502 with a configuration hint	Pass
TC-A09	Chat history containing a system role	422 (prompt-injection guard)	Pass
TC-A10	GET /tools	Same names and schemas the LLM receives	Pass
TC-A11	POST /tools/analyze_ecg/invoke	LSTM result returned with timing	Pass
TC-A12	Invoke unknown tool / bad arguments / unknown ecg_id / no model	404 / 422 / 404 / 503	Pass
TC-A13	POST /agent/chat with tool calls	Trace: LLM → tool → LLM	Pass
TC-A14	OpenAI fails, Gemini configured (scripted)	Answer from fallback; flagged in response	Pass
TC-I01	Start-up with real lifespan	Model loaded, Chroma collection created	Pass
TC-I02	Semantic search with real embeddings	ECG question → ECG document; baking question → cooking text	Pass
TC-I03	Restart the application	Knowledge base persists on disk	Pass
TC-I04	Record 233 uploaded as 250 Hz CSV	Beat count within 3%, V count within 15% of cardiologist labels	Pass
TC-I05	Upload with supplied R-peaks (record 100)	All peaks classified except edges; ≥ 95% N	Pass
TC-I06	Analyse response contract	All fields present; counts and percentages consistent	Pass
TC-I07	Upload and analyse a 5-minute strip over HTTP	< 5 s (measured ≈ 0.45 s)	Pass
TC-I08	Pure noise uploaded as an ECG	signal_quality.ok = false with a warning	Pass
TC-I09	Agent calls analyze_ecg and search on real services	Tool results contain the real model output and cited chunks	Pass

7.5 Live LLM Test Cases

These tests run the complete system with the real OpenAI gpt-4o-mini agent (temperature 0) and real Gemini 2.5 Flash calls, on held-out patients.

Table 12: Live end-to-end test cases with real LLMs

ID	Test case	Expected result	Status
TC-L01	GET /health	Main openai:gpt-4o-mini, fallback gemini:gemini-2.5-flash	Pass
TC-L02	Question about record 233	Agent calls analyze_ecg with the right id; answer quotes the model's V count or share; states not a diagnosis	Pass
TC-L03	Noise uploaded as ECG	Agent calls the tool and reports noise / poor quality	Pass
TC-L04	Which recordings are uploaded?	list_ecg_recordings called; both names in the answer	Pass
TC-L05	Multi-turn follow-up	Exact V count given in the second turn	Pass
TC-L06	Question answered from the knowledge base	search tool called; sources cited from the reference document	Pass

ID	Test case	Expected result	Status
TC-L07	ECG and guideline question together	Both tools called; sources cited	Pass
TC-L08	Tools API vs agent on the same ECG	Answer contains the N and V counts from the direct tool call	Pass
TC-L09	Step trace	LLM decides to call the tool, tool succeeds, LLM answers; token counts present	Pass
TC-L10	Gemini as main agent	Gemini calls analyze_ecg and answers	Pass
TC-L11	OpenAI key rejected	Gemini fallback answers; fallback_used = true	Pass

7.6 Mutation Testing

To check that the tests detect real faults, bugs were injected deliberately into the code [33] and the suite was re-run; every mutation was detected (Table below).

Table 13: Injected faults and the tests that detected them

Injected fault	Tests failed
RR features passed in the wrong order	Unit tests on RR features
Similarity score computed as distance instead of 1 − distance	Vector-store ranking tests
Tool result not returned to the LLM	Agent loop tests
API path ignores RR features	2 model tests
Signal-quality check disabled	4 tests (3 unit, 1 integration)
Rhythm features replaced by zeros at inference	7 of 14 performance tests (calibration, AUC, specificity, S detection, significance)

7.7 Defects Found by Testing

- **Patient leakage in validation (v1)**: a random beat split put training patients in the validation set. Fixed with patient-wise validation.
- **Over-prediction of fusion beats (v2)**: found by the held-out evaluation; fusion was removed from the trained classes.
- **Unhandled LLM errors**: an invalid key produced an HTTP 500; it now returns 502 with a hint.
- **Beats reported in pure noise**: an integration test showed that 20 s of Gaussian noise produced 39 "beats". A kurtosis-based signal-quality index was added; it flags all noise inputs and none of the 22 test recordings.
- **Edge-beat misalignment**: a consistency test exposed that segmentation drops edge beats; the training loader now records the R-peak of each kept beat.

CHAPTER 8: RESULTS AND DISCUSSION

8.1 Overall Performance on Unseen Patients

Table 14: Performance on the 22 DS2 test patients at annotated R-peaks (95% patient-bootstrap CI)

Model	Accuracy	Macro-F1	Balanced accuracy
Majority class (always N)	0.897	0.315	0.333
RR-only logistic regression	0.873	0.530	0.607
Conv-BiLSTM (this work)	0.945 [0.881–0.981]	0.650 [0.544–0.777]	0.630 [0.560–0.779]

Table 15: Per-class results on DS2

Class	Beats	Sensitivity [95% CI]	PPV [95% CI]	Specificity	F1
N	44,240	0.991 [0.984–0.997]	0.958 [0.904–0.990]	0.621	0.974
S	1,837	0.096 [0.006–0.498]	0.361 [0.069–0.514]	0.993	0.151
V	3,220	0.802 [0.604–0.926]	0.847 [0.543–0.970]	0.990	0.824

The model beats both baselines on macro-F1, and the lower bound of its confidence interval (0.544) is well above the majority baseline (Figure 6). Normal beats are recognised almost perfectly (Se 0.991), and ventricular beats are detected with sensitivity 0.802 and precision 0.847. Supraventricular beats are the main weakness, discussed in Section 8.5.

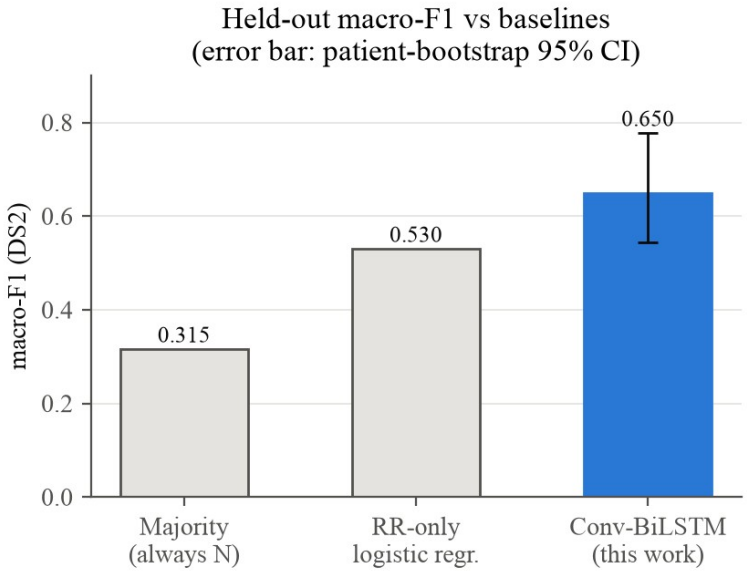


Figure 6: Held-out macro-F1 of the model and the baselines

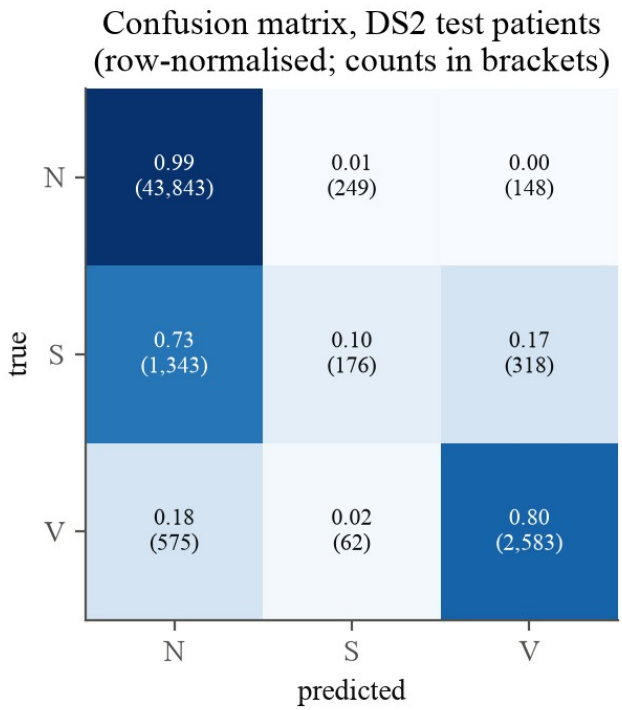


Figure 7: Confusion matrix on the DS2 test patients

8.2 End-to-End Performance

With the deployed R-peak detector instead of the annotations, 48,653 of 49,712 annotated beats are found (detection sensitivity 0.979, PPV 0.998, ±150 ms). Classification of the detected beats reaches accuracy 0.942 and macro-F1 0.642, with V sensitivity 0.805 and PPV 0.814. Performance therefore barely changes when the model runs on its own detected beats, which is the situation of a real user.

8.3 Calibration

The predicted probabilities are well calibrated: ECE is 0.0126, the Brier score 0.0924, and the reliability curve lies close to the diagonal (Figure 8). The ROC AUC is 0.9257 for N, 0.7638 for S and 0.9836 for V. Confidence is useful in practice: on the 91.5% of beats with confidence ≥ 0.9 the accuracy is 96.7%, so low-confidence beats can be highlighted for human review. The agent receives the confidence of every abnormal beat.

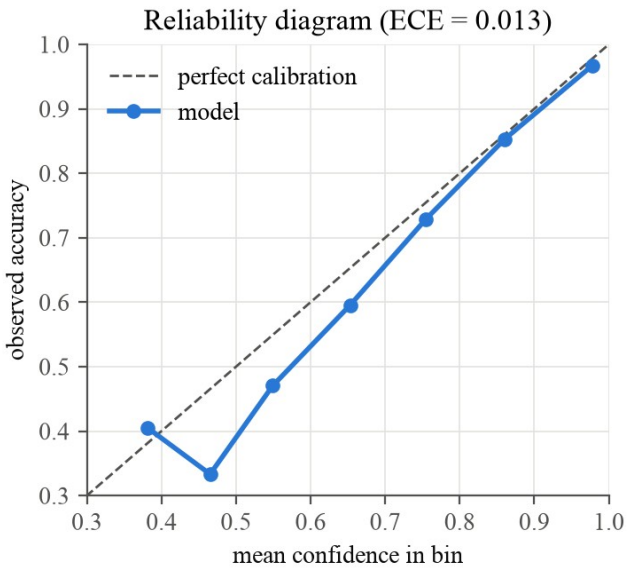


Figure 8: Reliability diagram of the predicted-class confidence on DS2

8.4 Computational Performance

Table 16: Speed and resources on an Intel i5-7200U laptop CPU without GPU

Measurement	Result
Inference, batch of 2,048 beats	0.447 ms per beat (2,237 beats/s)
Inference, single beat	78.01 ms (dominated by framework overhead)
Full analysis of a 30-minute record	1.31 s, about 1,373× faster than real time
API upload + analysis, 1-minute strip	160 ms (median of 5)
API upload + analysis, 5-minute strip	454 ms (median of 5)
Model size	49,571 parameters, 0.71 MB
Training time	8 epochs, about 9 minutes

8.5 Error Analysis

S beats are mostly missed (Se 0.096). 1,382 of the 1,837 S beats in DS2 come from record 232, a patient with sinus bradycardia and long pauses, in which atrial premature beats are not early relative to the patient's own rhythm, the main cue the model relies on. Without this record, accuracy is 0.972, macro-F1 0.740, and S sensitivity 0.382. The patient-level confidence interval for S sensitivity ([0.006–0.498]) reflects this dependence on individual patients.

V sensitivity also varies between patients: per record it ranges from 0.42 to 1.00. Only 0.33% of normal beats are wrongly reported as V, so false ventricular alarms are rare. Of the 388 fusion beats in DS2, 380 are labelled N, as expected for a class that was not trained.

Figure 9 shows ten seconds of record 233, a held-out patient. The model flags every ventricular beat with confidence above 0.95 and leaves the normal beats unflagged; the single S beat at 20.8 s is missed, illustrating the weakness described above.

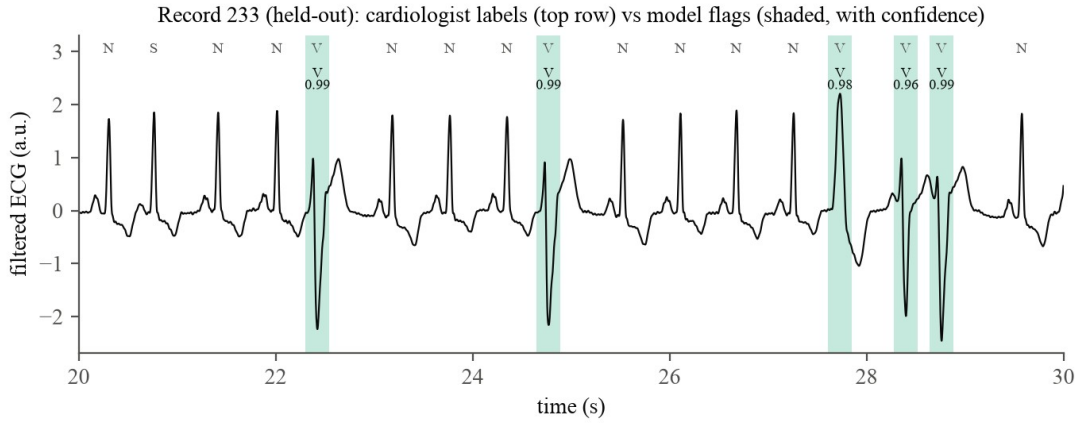


Figure 9: Model output on record 233 compared with cardiologist labels

8.6 Agent Behaviour with a Real LLM

With the real OpenAI model, a question such as “Is anything abnormal in this ECG? Give the numbers.” about 30 seconds of record 233 produced the following trace: the LLM decided to call analyze_ecg (about 2.0 s, 361 prompt and 24 completion tokens); the LSTM tool ran in about 0.1 s; the LLM then wrote the answer (about 7.2 s, 1,009 prompt and 531 completion tokens). The answer reported 36 normal and 13 ventricular beats, identical to a direct call of the tool through the Tools API, listed the abnormal beats with their times and confidences, and ended with the disclaimer. Most of the response time is spent in the LLM, not in the classifier.

Across the 11 live tests the agent always called the ECG tool instead of guessing, quoted the classifier's numbers, cited the knowledge base when asked factual questions, and recognised noise input from the signal-quality block. When the OpenAI key was invalid, Gemini answered the same question with the same tool calls.

8.7 Discussion

The results show that a small model trained on a laptop can reach useful performance on unseen patients for normal and ventricular beats, the most frequent and clinically most relevant ectopic class. They also confirm what the literature reports for the inter-patient setting [3], [13]: supraventricular beats are much harder to recognise on new patients than in intra-patient experiments. Reporting patient-level confidence intervals, baselines and a per-record breakdown makes this visible, where a single accuracy figure (94.5%) would hide it.

Integrating the model as an LLM tool keeps the measurement and the explanation separate. The classifier produces the numbers, the knowledge base supplies the reference text, and the LLM only combines them, with instructions to state the model's limitations, to mention its confidence and to disregard results flagged by the signal-quality check.

CHAPTER 9: LIMITATIONS AND ETHICAL CONSIDERATIONS

9.1 Limitations

- Low sensitivity for supraventricular beats; the model must not be used to rule out atrial ectopy.
- Fusion and paced beats are not recognised; recordings from patients with pacemakers are out of scope.
- Trained on a single lead (MLII) of a single database of 47 subjects recorded in 1975–1979; other leads, wearable devices and populations are untested.
- RR features use the recording's own rhythm as reference, so strips shorter than about 10 beats give less reliable results.
- Uploaded ECGs are kept in memory and are lost on restart; the service has no authentication.
- The quality of the agent's answers depends on the external LLM. It was exercised in 11 live tests with OpenAI and Gemini, but its explanations have not been evaluated by clinicians, and LLM providers may change model behaviour over time.

9.2 Ethical Considerations

- **Not a medical device.** Every analysis includes a disclaimer, and the agent is instructed to state that results are not a diagnosis.
- **Data privacy.** The MIT-BIH recordings are de-identified and openly licensed. ECGs uploaded by users are kept in memory only. The question, the conversation history and the tool results (heart rate, beat counts, abnormal beat times, retrieved document chunks) are sent to the LLM provider (OpenAI, or Google for the fallback); the raw ECG signal is not. A local model can be configured (LLM_PROVIDER=custom) to keep all data on-premises.
- **Transparency.** Limitations, failure cases and rejected model versions are documented in the model card and in this report.
- **Confidentiality.** The project uses only public data and open-source software and contains no confidential or customer information.

CHAPTER 10: CONCLUSION AND FUTURE WORK

10.1 Conclusion

This project delivered a complete, tested system for ECG beat classification and its use by an LLM agent. A Conv-BiLSTM of 49,571 parameters, trained on a laptop CPU, reaches 94.5% accuracy and a macro-F1 of 0.650 on 22 unseen patients, beating a rhythm-only baseline in a paired patient-level bootstrap, with well-calibrated probabilities and an end-to-end pipeline that analyses a 30-minute recording in about a second. The model is served by a FastAPI application with a persistent Chroma knowledge base. An OpenAI model is the agentic core that calls the classifier and the knowledge base as tools, with Gemini as automatic fallback, and the same tools are available through a separate REST API. 119 automated tests, checked by mutation testing, guard preprocessing, APIs, integration, accuracy, performance and the live agent.

10.2 Contributions

1. A shared preprocessing pipeline with patient-relative rhythm features and a kurtosis-based signal-quality check.
2. A strict inter-patient evaluation with baselines, patient-bootstrap confidence intervals, end-to-end detection, calibration and error analysis.
3. A deployable service in which an OpenAI agent, with a Gemini fallback, uses the model and a RAG knowledge base as tools, and a separate tools API that exposes the same functions without an LLM.
4. A test suite that includes accuracy floors, performance tests and integration tests on held-out patients.

10.3 Future Work

1. Improve S-beat detection with longer rhythm context (sequence-of-beats models) and patient-adaptive fine-tuning on a few labelled beats.
2. Train and validate on additional databases (e.g. INCART, the European ST-T database) and on wearable single-lead devices.
3. Add fusion and paced classes with more varied training data.
4. Add persistent storage, authentication and audit logging for the uploaded recordings.
5. Evaluate the agent's answers with clinicians and with automatic groundedness metrics.

REFERENCES

[1] G. B. Moody and R. G. Mark, "The impact of the MIT-BIH Arrhythmia Database," IEEE Engineering in Medicine and Biology Magazine, vol. 20, no. 3, pp. 45–50, 2001.

[2] A. L. Goldberger, L. A. N. Amaral, L. Glass, J. M. Hausdorff, P. C. Ivanov, R. G. Mark, J. E. Mietus, G. B. Moody, C.-K. Peng and H. E. Stanley, "PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals," Circulation, vol. 101, no. 23, pp. e215–e220, 2000.

[3] P. de Chazal, M. O'Dwyer and R. B. Reilly, "Automatic classification of heartbeats using ECG morphology and heartbeat interval features," IEEE Transactions on Biomedical Engineering, vol. 51, no. 7, pp. 1196–1206, 2004.

[4] ANSI/AAMI EC57:2012, Testing and Reporting Performance Results of Cardiac Rhythm and ST Segment Measurement Algorithms. Arlington, VA: Association for the Advancement of Medical Instrumentation, 2012.

[5] J. Pan and W. J. Tompkins, "A real-time QRS detection algorithm," IEEE Transactions on Biomedical Engineering, vol. BME-32, no. 3, pp. 230–236, 1985.

[6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[7] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," IEEE Transactions on Signal Processing, vol. 45, no. 11, pp. 2673–2681, 1997.

[8] S. Kiranyaz, T. Ince and M. Gabbouj, "Real-time patient-specific ECG classification by 1-D convolutional neural networks," IEEE Transactions on Biomedical Engineering, vol. 63, no. 3, pp. 664–675, 2016.

[9] U. R. Acharya, S. L. Oh, Y. Hagiwara, J. H. Tan, M. Adam, A. Gertych and R. S. Tan, "A deep convolutional neural network model to classify heartbeats," Computers in Biology and Medicine, vol. 89, pp. 389–396, 2017.

[10] A. Y. Hannun, P. Rajpurkar, M. Haghpanahi, G. H. Tison, C. Bourn, M. P. Turakhia and A. Y. Ng, "Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network," Nature Medicine, vol. 25, no. 1, pp. 65–69, 2019.

[11] Ö. Yildirim, "A novel wavelet sequence based on deep bidirectional LSTM network model for ECG signal classification," Computers in Biology and Medicine, vol. 96, pp. 189–202, 2018.

[12] V. Mondéjar-Guerra, J. Novo, J. Rouco, M. G. Penedo and M. Ortega, "Heartbeat classification fusing temporal and morphological information of ECGs via ensemble of classifiers," Biomedical Signal Processing and Control, vol. 47, pp. 41–48, 2019.

[13] G. Llamedo and J. P. Martínez, "Heartbeat classification using feature selection driven by database generalization criteria," IEEE Transactions on Biomedical Engineering, vol. 58, no. 3, pp. 616–625, 2011.

[14] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel and D. Kiela, "Retrieval-augmented generation for

knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems 33 (NeurIPS), 2020, pp. 9459–9474.

[15] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in International Conference on Learning Representations (ICLR), 2023.

[16] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in Advances in Neural Information Processing Systems 36 (NeurIPS), 2023.

[17] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proceedings of EMNLP-IJCNLP, 2019, pp. 3982–3992.

[18] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in Advances in Neural Information Processing Systems 33 (NeurIPS), 2020.

[19] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in International Conference on Learning Representations (ICLR), 2015.

[20] B. Efron and R. J. Tibshirani, An Introduction to the Bootstrap. New York: Chapman & Hall/CRC, 1993.

[21] C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, "On calibration of modern neural networks," in Proceedings of the 34th International Conference on Machine Learning (ICML), 2017, pp. 1321–1330.

[22] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in Proceedings of the 32nd International Conference on Machine Learning (ICML), 2015, pp. 448–456.

[23] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," Journal of Machine Learning Research, vol. 15, pp. 1929–1958, 2014.

[24] G. W. Brier, "Verification of forecasts expressed in terms of probability," Monthly Weather Review, vol. 78, no. 1, pp. 1–3, 1950.

[25] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[26] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2016, pp. 265–283.

[27] C. Xie, L. McCullum, A. Johnson, T. Pollard, B. Gow and B. Moody, "Waveform Database Software Package (WFDB) for Python," PhysioNet. [Online]. Available: https://github.com/MIT-LCP/wfdb-python

[28] Q. Li, R. G. Mark and G. D. Clifford, "Robust heart rate estimation from multiple asynchronous noisy sources using signal quality indices and a Kalman filter," Physiological Measurement, vol. 29, no. 1, pp. 15–32, 2008.

[29] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," Nature Methods, vol. 17, pp. 261–272, 2020.

[30] C. R. Harris et al., “Array programming with NumPy,” Nature, vol. 585, pp. 357–362, 2020.

[31] OpenAI, “Chat Completions API and function calling,” API reference. [Online]. Available: https://platform.openai.com/docs/api-reference

[32] Chroma, “Chroma: the open-source AI application database,” documentation. [Online]. Available: https://docs.trychroma.com

[33] Y. Jia and M. Harman, “An analysis and survey of the development of mutation testing,” IEEE Transactions on Software Engineering, vol. 37, no. 5, pp. 649–678, 2011.

[34] S. Ramírez, “FastAPI,” documentation. [Online]. Available: https://fastapi.tiangolo.com

[35] G. B. Moody and R. G. Mark, “MIT-BIH Arrhythmia Database” (version 1.0.0), PhysioNet, 2005. [Online]. Available: https://physionet.org/content/mitdb/1.0.0/

[36] Google, “Gemini API: OpenAI compatibility,” documentation. [Online]. Available: https://ai.google.dev/gemini-api/docs/openai

APPENDIX A: HOW TO REPRODUCE

All commands are run from the project root on Linux with Python 3.12.

```
python3 -m venv env && source env/bin/activate && pip install -r
requirements.txt
cp .env.example .env    # set OPENAI_API_KEY (and GEMINI_API_KEY for the
fallback)
jupyter nbconvert --to notebook --execute notebooks/01_ecg_analysis.ipynb
jupyter nbconvert --to notebook --execute
notebooks/02_lstm_training.ipynb
python scripts/evaluate_model.py      # reports/evaluation.md
python scripts/benchmark_model.py     # reports/performance.md
python scripts/test_report.py         # runs all tests,
reports/test_report.md
python scripts/make_figures.py        # docs/figures/
RUN_LLM_TESTS=1 pytest -m llm         # live tests with the real OpenAI
and Gemini APIs
uvicorn app.main:app                  # API docs at
http://localhost:8000/docs
python scripts/demo_agent.py          # tools API and agent trace against
the running server
```

Model artefacts (models/), MIT-BIH data (data/) and the vector store (chroma_db/) are generated by these steps and are not stored in version control. The full API reference with request and response examples is in docs/API.md of the project repository.

APPENDIX B: COMPLETE TEST CASE CATALOGUE

Generated from reports/junit.xml: 119 passed, 0 failed or skipped, of 119 tests.

Table 17: All automated test cases

#	Suite	Test case	Time (s)	Result
1	agent	tool schemas are valid	0.00	Pass
2	agent	registry roundtrip	0.00	Pass
3	agent	answer without tools	0.09	Pass
4	agent	agent runs ecg tool and feeds result back	0.03	Pass
5	agent	agent rag tool collects sources	0.03	Pass
6	agent	agent handles unknown ecg and bad arguments	0.03	Pass
7	agent	agent without model reports error	0.03	Pass
8	agent	agent stops after max steps	0.03	Pass
9	agent	provider endpoints resolve keys urls and default models	0.01	Pass
10	agent	missing provider key gives clear error	0.00	Pass
11	agent	fallback provider answers when main fails	0.03	Pass
12	agent	error propagates when fallback also fails	0.03	Pass
13	agent	temperature is sent only when configured	0.03	Pass
14	agent	trace records each llm decision and tool execution	0.03	Pass
15	agent	llm gets validation errors for bad arguments	0.03	Pass
16	api	health	0.09	Pass
17	api	openapi lists all routes	0.07	Pass
18	api	upload search delete document	0.05	Pass
19	api	reingest replaces previous chunks	0.04	Pass
20	api	upload rejects unsupported and empty	0.04	Pass
21	api	search validates top k	0.03	Pass
22	api	ecg upload json and analyze	0.04	Pass
23	api	ecg upload csv skips header	0.04	Pass
24	api	ecg upload validation	0.04	Pass
25	api	ecg analyze unknown id	0.03	Pass
26	api	ecg analyze without model returns 503	0.04	Pass
27	api	chat end to end with tool call	0.04	Pass
28	api	chat llm failure returns 502 with hint	0.04	Pass
29	api	chat rejects unknown ecg and bad history	0.04	Pass
30	api	tools catalogue matches what the llm receives	0.04	Pass
31	api	invoke analyze ecg tool directly	0.04	Pass
32	api	invoke search and list tools directly	0.04	Pass
33	api	invoke tool error codes	0.05	Pass
34	api	agent config endpoint	0.04	Pass
35	api	agent chat returns step trace and chat alias still works	0.04	Pass
36	api_integration	startup loads real model and store	3.84	Pass
37	api_integration	ingest reference document and retrieve relevant chunks	0.96	Pass
38	api_integration	semantic search prefers matching document	0.93	Pass
39	api_integration	knowledge base persists across restart	0.71	Pass
40	api_integration	csv upload at 250hz matches cardiologist labels	1.42	Pass
41	api_integration	json upload with supplied r peaks	0.20	Pass
42	api_integration	analyze response contract	0.18	Pass
43	api_integration	analyze latency	0.97	Pass
44	api_integration	noise only input is flagged as unreliable	0.11	Pass

#	Suite	Test case	Time (s)	Result
45	api_integration	real ecg passes signal quality	0.21	Pass
46	api_integration	agent uses real model and knowledge base	0.49	Pass
47	ingest	extract text txt and md	0.00	Pass
48	ingest	extract text pdf	0.00	Pass
49	ingest	extract text rejects unsupported	0.00	Pass
50	ingest	chunk text respects size and overlap	0.00	Pass
51	ingest	chunk text drops blank input	0.00	Pass
52	llm_live	health reports openai main and gemini fallback	1.24	Pass
53	llm_live	agent analyzes uploaded ecg with the lstm	4.90	Pass
54	llm_live	noise recording is flagged not interpreted	6.25	Pass
55	llm_live	list recordings tool	2.17	Pass
56	llm_live	multi turn follow up uses history	4.70	Pass
57	llm_live	rag answer is grounded in the knowledge base	3.37	Pass
58	llm_live	combined ecg and guideline question uses both tools	7.84	Pass
59	llm_live	gemini as main agent calls tools	3.88	Pass
60	llm_live	falls back to gemini when openai rejects the key	5.58	Pass
61	llm_live	tool called directly matches what the agent received	4.61	Pass
62	llm_live	agent trace shows the full flow	5.78	Pass
63	model	meta matches preprocessing and has no patient leakage	0.00	Pass
64	model	predictions are deterministic probabilities	1.94	Pass
65	model	api pipeline matches training pipeline	0.85	Pass
66	model	regular rhythm record	0.52	Pass
67	model	detects ventricular ectopy	0.37	Pass
68	model	invariant to gain and dc offset	0.41	Pass
69	model	robust to lower sampling rate	0.72	Pass
70	model	robust to moderate noise	0.72	Pass
71	model	all test patients pass signal quality	4.32	Pass
72	model	short and flat inputs do not crash	0.13	Pass
73	model	ds2 accuracy floors	26.46	Pass
74	model	beats majority and rr only baselines	0.00	Pass
75	model	end to end detection and classification	26.37	Pass
76	model_performance	model is small enough for cpu deployment	0.00	Pass
77	model_performance	batched inference throughput	1.52	Pass
78	model_performance	full record analysed much faster than real time	1.35	Pass
79	model_performance	predictions do not depend on batch size	3.44	Pass
80	model_performance	memory stays bounded on repeated analysis	3.75	Pass
81	model_performance	probabilities are well calibrated	25.77	Pass
82	model_performance	roc auc per class	0.00	Pass
83	model_performance	higher confidence means higher accuracy	0.00	Pass
84	model_performance	few normal beats raised as ventricular	0.00	Pass
85	model_performance	v specificity	0.00	Pass
86	model_performance	s detection outside record 232	0.00	Pass
87	model_performance	v detected in most patients	0.01	Pass
88	model_performance	macro f1 ci lower bound beats baselines	0.00	Pass
89	model_performance	paired bootstrap vs rr baseline	5.82	Pass
90	preprocessing	beat window constants	0.00	Pass
91	preprocessing	aami mapping covers all classes	0.00	Pass
92	preprocessing	resample length[128]	0.00	Pass
93	preprocessing	resample length[250]	0.00	Pass

#	Suite	Test case	Time (s)	Result
94	preprocessing	resample length[500]	0.00	Pass
95	preprocessing	resample length[1000]	0.00	Pass
96	preprocessing	resample noop at target fs	0.00	Pass
97	preprocessing	bandpass removes baseline wander keeps qrs band	0.00	Pass
98	preprocessing	normalize beat	0.00	Pass
99	preprocessing	normalize flat beat is finite	0.00	Pass
100	preprocessing	detect r peaks on synthetic ecg[0.5]	0.05	Pass
101	preprocessing	detect r peaks on synthetic ecg[0.8]	0.04	Pass
102	preprocessing	detect r peaks on synthetic ecg[1.2]	0.02	Pass
103	preprocessing	detect r peaks after resampling from 250hz	0.01	Pass
104	preprocessing	rr features regular rhythm	0.00	Pass
105	preprocessing	rr features are heart rate independent	0.00	Pass
106	preprocessing	rr features flag premature beat	0.00	Pass
107	preprocessing	rr features degenerate inputs	0.00	Pass
108	preprocessing	segment beats skips edges	0.00	Pass
109	preprocessing	segment beats empty	0.00	Pass
110	preprocessing	signal quality accepts synthetic ecg	0.03	Pass
111	preprocessing	signal quality rejects non ecg[gaussian]	0.01	Pass
112	preprocessing	signal quality rejects non ecg[sine]	0.00	Pass
113	preprocessing	signal quality rejects non ecg[flat]	0.00	Pass
114	preprocessing	signal quality is scale invariant	0.05	Pass
115	vector_store	add search ranks relevant chunk first	0.33	Pass
116	vector_store	data persists across restarts	0.35	Pass
117	vector_store	reingest upserts and source filter and delete	0.50	Pass
118	vector_store	search empty collection and large top k	0.30	Pass
119	vector_store	refuses mismatched embedding model	0.35	Pass